`PySensemakr`: Sensitivity Analysis Tools for OLS in Python

10 February 2026

Summary

`PySensemakr` is a Python package that implements a suite of sensitivity analysis tools that extends the traditional omitted variable bias (OVB) framework for linear regression models, as developed in Cinelli and Hazlett (2020). The package allows researchers to (i) compute sensitivity statistics for routine reporting, such as the partial R^2 of the treatment with the outcome and the robustness value, (ii) derive bounds on the strength of unobserved confounders using observed covariates as benchmarks, and (iii) produce sensitivity contour plots and extreme scenario plots that provide a visual summary of the sensitivity of causal estimates to unobserved confounding. `PySensemakr` is built on top of `statsmodels` (Seabold and Perktold 2010) and requires only standard regression output, without further assumptions on the functional form of the treatment assignment mechanism or on the distribution of the unobserved confounders. The goal of `PySensemakr` is to make sensitivity analysis an accessible, routine part of empirical research in Python, thereby enabling a more transparent and disciplined discussion regarding the causal interpretation of regression estimates.

Statement of Need

The most common strategy for drawing causal inferences from observational data is to adjust for observed covariates in a regression model, while making the untestable assumption that there are no unobserved confounders. Since this assumption cannot be verified with the data at hand, sensitivity analysis—which asks how strong unobserved confounders would need to be in order to change the conclusions of a study—plays a central role in determining the credibility of causal claims based on observational data.

Cinelli and Hazlett (2020) proposes a modern suite of sensitivity analysis tools based on a partial R^2 parameterization of omitted variable bias. These tools (i) do not require assumptions on the functional form of the treatment assignment mechanism nor on the distribution of the unobserved confounders, (ii) naturally handle multiple confounders, possibly acting non-linearly, (iii) exploit expert knowledge to bound sensitivity parameters using observed covariates as benchmarks, and (iv) can be easily computed using only standard regression output. The methodology thus lends itself naturally to *routine reporting*—the practice of including sensitivity statistics alongside standard regression tables, much like one would report p-values or confidence intervals.

While `sensemakr` implementations already exist for R (Cinelli, Ferwerda, and Hazlett 2024) and Stata, Python has lacked an equivalent tool, despite being one of the most widely used languages for data analysis across the social sciences, biomedical research, and industry. `PySensemakr` fills this gap, bringing the full `sensemakr` methodology to the Python ecosystem.

State of the Field

The R version of `sensemakr` (Cinelli, Ferwerda, and Hazlett 2024) has been widely adopted across disciplines—including political science, economics, epidemiology, and education—and has been cited in thousands of empirical studies. A Stata version is also available. `PySensemakr` is a dedicated, full-featured Python implementation of the complete `sensemakr` methodology for OLS regression. It provides the full

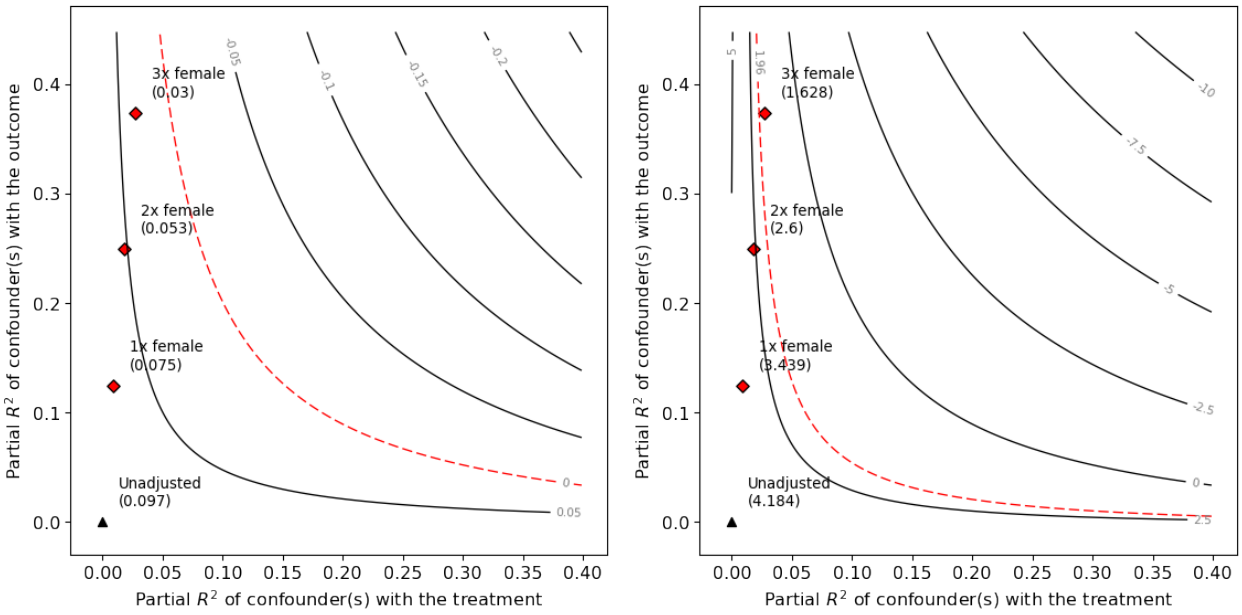
suite of sensitivity statistics (partial R^2, robustness values RV_q and $RV_{q,\alpha}$), the complete benchmarking apparatus, all bias-adjustment functions, and all visualization tools (contour plots and extreme scenario plots), with an API that closely mirrors the R version. `PySensemakr` brings this same dedicated toolkit to the Python ecosystem, lowering the barrier for researchers who wish to incorporate formal sensitivity analysis into Python-based regression workflows.

Software Design

`PySensemakr` takes a fitted `statsmodels OLSResults` object as input and computes all relevant sensitivity statistics from the standard regression output. The package is organized into the following modules:

- **`main`**: The `Sensemakr` class serves as the primary interface. It takes a fitted model, a treatment variable, and optional benchmark covariates, and computes sensitivity statistics and bounds. It provides `summary()` and `plot()` methods for reporting results.
- **`sensitivity_statistics`**: Functions for computing the partial R^2 of the treatment with the outcome, the robustness value RV_q (the minimum confounding strength required to reduce the estimate by a fraction q), and the robustness value $RV_{q,\alpha}$ (accounting for statistical significance).
- **`sensitivity_bounds`**: Functions for computing bounds on the bias-adjusted estimates, using observed covariates as benchmarks for the plausible strength of unobserved confounders.
- **`sensitivity_plots`**: Functions for producing contour plots of bias-adjusted estimates and t-values as functions of the sensitivity parameters $R^2_{D\sim Z|\mathbf{X}}$ and $R^2_{Y\sim Z|\mathbf{X},D}$, as well as extreme scenario plots.
- **`bias_functions`**: Core functions for computing the bias, bias-adjusted estimates, standard errors, and t-values.

?? illustrates the contour plot output for the example dataset of Hazlett (2020), in which the estimate and the t-value of the treatment effect are displayed as functions of the hypothetical confounding strength. Figure 1 shows the corresponding extreme scenario plot.



The following code illustrates a typical workflow:

```
import sensemakr as smkr
import statsmodels.formula.api as smf

# Load data and fit regression model
darfur = smkr.load_darfur()
```

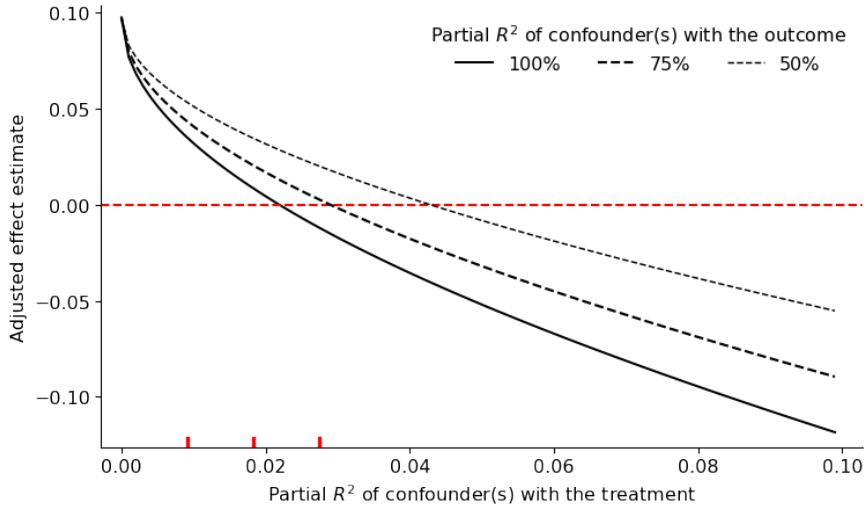


Figure 1: Extreme scenario plot for the Darfur data, showing the bias-adjusted estimate under the worst-case scenario for confounders of varying strength.

```
model = smf.ols(formula='peacefactor ~ directlyharmed + age + farmer_dar + '
                'herder_dar + pastvoted + hhsize_darfur + female + village',
                data=darfur).fit()

# Run sensitivity analysis
sense = smkr.Sensemakr(model=model,
                       treatment="directlyharmed",
                       benchmark_covariates=["female"],
                       kd=[1, 2, 3])

# Print and plot results
sense.summary()
sense.plot()
```

Research Impact

The sensitivity analysis methodology implemented in `PySensemakr` (Cinelli and Hazlett 2020) has had broad impact across the empirical sciences. The original theoretical paper has been cited over 1,800 times, and the R implementation of `sensemakr` (Cinelli, Ferwerda, and Hazlett 2024) has been used in published studies spanning political science, economics, epidemiology, sociology, education, and environmental science, among other fields. By providing a Python implementation, `PySensemakr` extends the reach of these tools to the large and growing community of researchers and data scientists who use Python as their primary computing environment, thereby helping to make formal sensitivity analysis a routine component of empirical research workflows.

AI Usage Disclosure

Generative AI tools were used to assist with code maintenance tasks (e.g., updating deprecated API calls for compatibility with newer versions of dependencies). All AI-generated changes were reviewed and tested by the authors.

Acknowledgements

We thank Chad Hazlett and Jeremy Ferwerda for their contributions to the theoretical development and to the R and Stata implementations of `sensemakr`, which served as the foundation for this Python package.

References

Cinelli, Carlos, Jeremy Ferwerda, and Chad Hazlett. 2024. "Sensemakr: Sensitivity Analysis Tools for OLS in r and Stata." *Journal of Statistical Software* 111 (4): 1–33. https://doi.org/10.18637/jss.v111.i04.

Cinelli, Carlos, and Chad Hazlett. 2020. "Making Sense of Sensitivity: Extending Omitted Variable Bias." *Journal of the Royal Statistical Society: Series B (Statistical Methodology)* 82 (1): 39–67. https://doi.org/10.1111/rssb.12348.

Hazlett, Chad. 2020. "Angry or Weary? How Violence Impacts Attitudes Toward Peace Among Darfurian Refugees." *Journal of Conflict Resolution* 64 (5): 844–70. https://doi.org/10.1177/0022002719879217.

Seabold, Skipper, and Josef Perktold. 2010. "Statsmodels: Econometric and Statistical Modeling with Python." In *Proceedings of the 9th Python in Science Conference*. https://doi.org/10.25080/Majora-92bf1922-011.